TECHNICAL TRAINING COURSE

JEE Entities and Complex Content

JAX-RS 2.x · JAXB 2.3

- Entity Providers · Built-in Providers
- Working with XML · JAXB Annotations
- Schema-First Development · XXE Security

Java 11 LTS | Tomcat 9.0 | JAX-RS 2.1 | JAXB 2.3

COURSE AGENDA

4 Sections | Entity Providers, XML & Schema-Driven Content

01 Entity Providers

02 Built-in Entity Providers

03 Working with XML

04 Driving XML from Schema

SECTION 1

Entity Providers — The Serialisation Extension Point

Every HTTP body ↔ Java object conversion is delegated to an entity provider. Two interfaces, two directions:

MessageBodyWriter<T> — Java → HTTP Body

Called during response serialisation.
The runtime calls isWriteable(), then writeTo().

MessageBodyReader<T> — HTTP Body → Java

Called during request deserialisation.
The runtime calls isReadable(), then readFrom().

Provider Selection — in priority order:

1. More specific media type wins (application/json beats application/*)
2. User-registered providers outrank built-in providers
3. Among equals, registration order breaks ties

SECTION 1

Custom MessageBodyWriter — CSV Example

```
@Provider
@Produces("text/csv")
public class ProductCsvWriter implements MessageBodyWriter<Product> {

    @Override
    public boolean isWriteable(Class<?> type, Type genericType,
                               Annotation[] annotations, MediaType mediaType) {
        return Product.class.isAssignableFrom(type)
            && mediaType.isCompatible(new MediaType("text","csv"));
    }

    @Override public long getSize(...) { return -1; }  // deprecated

    @Override
    public void writeTo(Product p, Class<?> type, Type genericType,
                        Annotation[] annotations, MediaType mediaType,
                        MultivaluedMap<String,Object> headers,
                        OutputStream entityStream) throws IOException {
        PrintWriter pw = new PrintWriter(
            new OutputStreamWriter(entityStream, StandardCharsets.UTF_8));
        pw.println("id,name,category,price");
        pw.printf("%s,%s,%s,%.2f%n", p.getId(), p.getName(),
                  p.getCategory(), p.getPrice());
        pw.flush();
        // NEVER close entityStream — the container owns it
    }
}
```

SECTION 2

Built-in Entity Providers

JAX-RS 2.1 mandates these providers — no additional dependencies required:

Java Type	Media Types	Notes
`byte[]`	`application/octet-stream, */*`	Raw binary — any media type
`String`	`text/*, */*`	Plain text — read/write
`InputStream`	`application/octet-stream, */*`	Memory-efficient streaming
`Reader`	`text/*, */*`	Character stream
`File`	`application/octet-stream, */*`	Streams file bytes directly
`StreamingOutput`	`application/octet-stream, */*`	Lazy callback writing
`MultivaluedMap`	`application/x-www-form-urlencoded`	Form fields as map
`@XmlRootElement class`	`application/xml, text/xml`	JAXB — see Section 3

SECTION 2

Configuring the Shared JAXBContext — ContextResolver<JAXBContext>

```
@Provider
public class JaxbContextResolver implements ContextResolver<JAXBContext> {

    private final JAXBContext ctx;
    public JaxbContextResolver() throws JAXBException {                    // LocalDate, Instant, etc.
        ctx = JAXBContext.newInstance(Order.class, // "2024-06-15" not 1718409600000
        .disable(SerializationFeature.INDENT_OUTPUT)              // compact production output
        .configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false) // tolerant
        .setSerializationInclusion(JsonInclude.Include.NON_NULL);// omit null fields

    @Override
    public JAXBContext getContext(Class<?> type) { return ctx; }
}
```

- Register as @Provider — auto-discovered by Jersey at startup
- JAXBContext is thread-safe — create ONCE in constructor or static field
- Supplies a pre-warmed context covering all domain classes
- Return type-specific mappers from getContext() for fine-grained control

SECTION 3

JAXB Annotation Quick Reference

`@XmlRootElement`

Declares a class as a JAXB root element — required for built-in JAX-RS provider

`@XmlType`

Sets element order (propOrder) and the XML Schema type name

`@XmlAccessorType`

FIELD (bind fields) or PROPERTY (bind getters). Use FIELD.

`@XmlElement`

Maps a field to a named child element. Use name= and required=true.

`@XmlAttribute`

Maps a field to an XML attribute on the opening tag

`@XmlElementWrapper`

Wraps a collection in a parent element: <lines><line>...</line></lines>

`@XmlTransient`

Excludes a field from XML output entirely

`@XmlJavaTypeAdapter`

Wires a custom XmlAdapter for unsupported types (e.g., LocalDate)

SECTION 3

Producing XML and JSON from One Class

```java
@XmlRootElement(name = "order",
  namespace = "http://example.com/shop/orders")
@XmlType(propOrder = {"customerName",
    "orderDate","lines","totalAmount"})
@XmlAccessorType(XmlAccessType.FIELD)
public class Order {

  @XmlAttribute(name="orderId", required=true)
  private String orderId;

  @XmlElement(required=true)
  @XmlJavaTypeAdapter(LocalDateAdapter.class)
  private LocalDate orderDate;

  @XmlElementWrapper(name="lines")
  @XmlElement(name="line")
  private List<OrderLine> lines;

  @XmlTransient
  private String internalRef;
}
```

```java
// Dual output resource
@GET @Path("{id}")
@Produces({
  // APPLICATION_XML only in this module
  APPLICATION_XML
})
public Response get(
   @PathParam("id") String id){
  Order o = repo.findById(id);
  return Response.ok(o).build();
}

// Accept: application/json
//   → JAXB provider marshals to XML

// Accept: application/xml
//   → Built-in JAXB
```

SECTION 3

Namespace Handling — @XmlSchema package-info

```
// src/main/java/com/example/model/package-info.java
@XmlSchema(
    namespace = "http://example.com/shop/orders",
    elementFormDefault = XmlNsForm.QUALIFIED
)
package com.example.model;
import javax.xml.bind.annotation.XmlSchema;
import javax.xml.bind.annotation.XmlNsForm;
```

This single file applies the namespace to ALL classes in com.example.model — no repetition needed.

Option A — @XmlRootElement(namespace=...) — Namespace on each class individually. Verbose but explicit.

Option B — package-info.java with @XmlSchema — Namespace applied to every class in the package. Preferred for large models.

elementFormDefault=QUALIFIED — All child elements also carry the namespace, not just the root. Required for schema-conformant output.

SECTION 4

Schema-First vs Java-First

Aspect	Java-First	Schema-First
Starting point	Java class + @XmlRootElement	XSD file (the contract)
Tooling	schemagen	xjc / jaxb2-maven-plugin
Type safety	Annotations can drift	Generated classes always match schema
Interoperability	Schema is derived, imprecise	Schema is the authoritative contract
Best for	Rapid prototyping, internal APIs	Partner APIs, regulated domains

SECTION 4

Generating Binding Classes with xjc

```
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>jaxb2-maven-plugin</artifactId>
  <version>3.1.0</version>
  <executions>
    <execution>
      <goals><goal>xjc</goal></goals>
    </execution>
  </executions>
  <configuration>
    <sources><source>src/main/xsd</source></sources>
    <packageName>com.example.shop.generated</packageName>
  </configuration>
</plugin>
```

Workflow

1. Write order-schema.xsd in src/main/xsd/
2. Run: mvn generate-sources
3. Classes appear in target/generated-sources/jaxb/
4. Mark directory as Generated Sources Root in IDE
5. Use ObjectFactory to create instances
6. Optionally create bindings.xjb to rename classes

SECTION 4

Schema Validation + XXE Security

XXE: Attackers embed DOCTYPE to read /etc/passwd or trigger SSRF. Disable external entities on ALL XMLInputFactory instances.

```
// Validating MessageBodyReader
@Provider @Consumes(APPLICATION_XML)
public class ValidatingOrderReader
        implements MessageBodyReader<OrderType> {

    static { CTX = JAXBContext.newInstance(OrderType.class);
             SCHEMA = SchemaFactory.newInstance(
               W3C_XML_SCHEMA_NS_URI).newSchema(
               getResource("/xsd/order-schema.xsd")); }

    @Override public OrderType readFrom(...,
            InputStream entityStream) throws IOException {
        Unmarshaller u = CTX.createUnmarshaller();
        u.setSchema(SCHEMA);  // enforce schema
        u.setEventHandler(event -> {
            if(event.getSeverity()>=ERROR)
                throw new RuntimeException(event.getMessage());
            return true; });
        return u.unmarshal(...).getValue();
    }
}
```

```
// XXE Prevention
// ALWAYS set these
// before parsing XML
// from untrusted sources

XMLInputFactory xif =
  XMLInputFactory.newInstance();

xif.setProperty(
  IS_SUPPORTING_
  EXTERNAL_ENTITIES,
  false);

xif.setProperty(
  SUPPORT_DTD,
  false);
```

COURSE SUMMARY

Key Takeaways

01 MessageBodyWriter/Reader are the extension points for all entity serialisation. isWriteable/isReadable guards selection.

02 Built-in providers cover byte[], String, InputStream, File, StreamingOutput, and @XmlRootElement classes.

03 User-registered providers always outrank built-in providers of equal media-type specificity.

04 ContextResolver<JAXBContext> supplies a pre-warmed shared context; javax.xml.bind.* (NOT jakarta.*) throughout.

05 @XmlRootElement is required for the built-in JAXB provider. @XmlElementWrapper wraps collections neatly.

06 Schema-first (xjc) produces type-safe, schema-conformant classes. Use ObjectFactory, not new, to create instances.

07 Always disable IS_SUPPORTING_EXTERNAL_ENTITIES and SUPPORT_DTD on XMLInputFactory when accepting client XML.

Thank You

Questions & Lab Time

Lab exercises:

- Lab 1: Custom MessageBodyWriter — CSV provider
- Lab 2: ContextResolver<JAXBContext> — shared JAXB context configuration
- Lab 3: JAXB annotations — dual JSON/XML output
- Lab 4: Schema-first with xjc — validating reader